

SpecuNode

Speculative execution for agent graphs, with a store buffer: what happens when an AI agent runs ahead of its own model, and how to make sure nothing it guessed reaches the world.

Every number in this document is read from a committed file in `bench/results/`. Nothing here is typed in by hand. A section whose results file is missing says so rather than being omitted.

1. How much speculation a real corpus exposes

Corpus: `nebius/SWE-rebench-openhands-trajectories` (huggingface), 300 trajectories, 19484 tool calls. Effect classes come from a committed hand-written table per tool name; anything absent from it is a `WRITE`, exactly as in the runtime.

Measure	Mean [95% CI]
Reads - speculable under any policy	0.0445 [0.0423, 0.0469]
Calls opening a new model turn	1.0000 [1.0000, 1.0000]
Args referencing a prior result	0.1957 [0.1875, 0.2040]
PASTE speculable span (calls)	0.0462 [0.0438, 0.0487]
Steps PASTE skips, SpecuNode stages	0.9555 [0.9531, 0.9577]
SpecuNode span past a write (calls)	0.0000 [0.0000, 0.0000]
Model turn consuming a write result	0.9547 [0.9523, 0.9569]

On this corpus, running ahead past a write buys nothing at all. The span is 0.0000, with a bootstrap interval that does not move off zero. Every tool call in these trajectories opens a new model turn, and a staged write blocks the next turn because that turn would have to contain a placeholder where the real result belongs. 95.5% of the corpus is that shape.

The half that is not zero: PASTE excludes a tool with side effects from speculation entirely, so it can speculate on 4.5% of steps. SpecuNode stages a write instead of refusing it, covering the other 95.5%. That is an upper bound on opportunity, not a speedup, realisable only where the predictor is right. The nearest measured proxy is signature accuracy at 53.5% top-1, which compares tool names and argument keys but not argument values, and is therefore an upper bound on acceptance rather than a measurement of it top-1. Neither number means anything alone.

The acceptance rate, measured

The signature figure is an upper bound; this is the quantity itself. The tier-1 predictor, graded by the runtime's own gate on exact canonical equality of argument values, over all 300 trajectories joined to their argument values, leave-one-trajectory-out. **within_turn** is the policy the runtime runs; **across_turns** is what carrying a guess into the next model turn would be worth.

Measure	within_turn	across_turns
Steps graded	19184	19184
Guesses offered	0.9533	0.9794
Acceptance rate, pooled	0.0000	0.0002
Acceptance rate, by trajectory	0.0000 [0.0000, 0.0000]	0.0002 [0.0000, 0.0004]

Confirmed guesses	0	3
Signature top-1, same steps	0.5219	0.5339
Squashed at a turn boundary	18288	0

The tier-1 acceptance rate on this corpus is 0.0002 at best and 0.0000 under the runtime's own policy. The index knows which tool comes next 53.4% of the time; the gate needs the exact command, path or thought. Every argument value of a call has already appeared earlier at 9.8% of steps, which is the ceiling for any predictor that copies values out of history, and this one is far below it. Anything above the ceiling needs a predictor that generates values, and that has not been measured.

2. What beats it

#	Strategy	Outcome	Measured
7.1	misdeclared tool (READ that writes)	beats it	effects_from_squashed_branch=1; leak_rate=1.0
7.2	read with upstream side effects	beats it	reads_from_squashed_branches=5; reads_issued=10
7.3	stale reads under contention	beats it	detected_stale=1; reads=2; undetectable_fraction=0.5
7.4	duplicate delivery of a non-idempote	beats it	delivered=2; duplicates=1; intended=1
7.5	return-value laundering	beats it	cases=9; caught=6; miss_rate=0.333
7.6	prompt-injected tool call	beats it	dispatched_when_only_the_drafter_predicted_it=0; dis
7.7	drafter poisoning	held	alpha_window=4; charges_staged_then_discarded=4; con
7.8	replay under model drift	held	both_caught_at_step_0=1; first_divergence_step_after
7.9	staging an irreversible effect	beats it	barrier_by_default=1; staged_when_enabled=1
7.10	asynchronous side effect behind a RE	beats it	effects_landing_after_the_squash=1; leaked_effects_p

3. Demo 1 - speculation without a store buffer

Runtime	Runs	Mispredictions	Reached world	From squashed branches
naive-parallel	50	10	60	10
specunode	50	10	50	0
sequential	50	0	50	0

The last column, and the fact that the second and third rows agree.

4. Invariants under load, and what the runtime costs

Matrix	Runs	Leaks	Duplicates	Equivalence failures
Chaos	12	0	0	0
Concurrency	20	0	0	0

Journaling and classification cost 6.645 ms per step (19.934 ms per run over 3 steps). That is 87.7% of wall clock here, and the percentage is the misleading half: a scripted model answers instantly, so this is the worst case for the ratio.

5. Wall-clock latency, against a real model

The 270 runs on file were taken before two faults in their arms were found -- the baseline already issued reads early, and the reads-only arm was given no predictor -- so their savings are not quoted. Section 6's 0 ms rung is the corrected measurement; re-running this table is pending.

6. How slow a tool has to be

Tool latency	Workload	Early issue only	Early issue + guessing
--------------	----------	------------------	------------------------

0 ms	ops_agent	+5.9% [-4.0%, +18.8%]	-1.8% [-17.2%, +12.9%]
0 ms	research_agent	+0.8% [-7.8%, +8.6%]	+0.7% [-6.9%, +7.8%]
0 ms	support_agent	-2.7% [-14.3%, +7.5%]	+2.6% [-5.9%, +10.7%]
500 ms	ops_agent	+11.8% [+7.8%, +15.5%]	+14.4% [+12.1%, +16.7%]
500 ms	research_agent	+1.1% [-3.2%, +5.2%]	-0.8% [-4.9%, +3.2%]
500 ms	support_agent	-0.1% [-4.0%, +3.6%]	-0.7% [-5.1%, +3.5%]
2000 ms	ops_agent	+16.7% [+15.1%, +18.2%]	+15.6% [+13.8%, +17.4%]
2000 ms	research_agent	+0.9% [-1.1%, +2.8%]	-1.2% [-4.2%, +1.4%]
2000 ms	support_agent	-2.7% [-5.7%, -0.2%]	-0.3% [-1.9%, +1.3%]

Only the workload that hands its model turn to the runtime can gain anything; the other two call the model themselves. Where it gains, the whole of the saving is tier-0 early issue, and guessing adds nothing the interval can resolve.

7. What a guess is worth

Tool latency	Accuracy	Guess reads and writes	Guess reads only
500 ms	0.00	-0.2% [-0.3%, -0.0%]	-0.4% [-0.9%, +0.0%]
500 ms	0.25	-0.3% [-0.9%, +0.1%]	-0.1% [-0.4%, +0.1%]
500 ms	0.50	-0.2% [-0.4%, +0.1%]	-0.1% [-0.5%, +0.1%]
500 ms	0.75	-0.9% [-1.1%, -0.7%]	-0.8% [-1.1%, -0.5%]
500 ms	1.00	-1.3% [-2.1%, -0.7%]	-1.3% [-2.0%, -0.8%]
2000 ms	0.00	+0.1% [-0.1%, +0.4%]	+0.1% [-0.2%, +0.3%]
2000 ms	0.25	+2.4% [+0.8%, +3.9%]	+2.5% [+0.9%, +4.2%]
2000 ms	0.50	+4.2% [+2.9%, +5.0%]	+4.2% [+3.0%, +5.0%]
2000 ms	0.75	+4.8% [+4.5%, +5.1%]	+4.7% [+4.4%, +5.0%]
2000 ms	1.00	+4.7% [+4.4%, +5.0%]	+4.8% [+4.5%, +5.1%]

A guesser of controlled accuracy, against early issue alone, on a stand-in that streams 500 ms per block. A correct guess can run at most one block ahead; a wrong one is squashed and costs no wall clock; and guessing writes as well as reads adds nothing, because a staged write cannot be sent before its branch retires.

8. The acceptance rate of a draft model

claude-haiku-4-5, graded by the runtime's own gate on 1000 sampled steps: acceptance 6.9% [5.4%, 8.6%], right tool 47.3%. Against a tier-1 acceptance rate that is effectively zero, a draft model is the only predictor here that gets anything right -- and section 7 is what a hit is worth when it does.

9. When the model is the slow part

Fewer replies, and replies side by side, against a stand-in model whose reply takes 1500 ms before its first block and 300 ms per block -- settings, not measurements. The reply and in-flight counts do not depend on them.

Tool latency	One call per reply	Every independent call at once	Saving
0 ms	9 replies, 19200 ms	4 replies, 9968 ms	+48.1% [+47.6%, +49.0%]
300 ms	9 replies, 20246 ms	4 replies, 11198 ms	+44.7% [+44.6%, +44.8%]

Tool latency	Nodes one after another	Side by side	Saving
0 ms	1 in flight, 6350 ms	3 in flight, 2153 ms	+66.1% [+65.7%, +66.3%]
300 ms	1 in flight, 8467 ms	3 in flight, 3656 ms	+56.8% [+56.7%, +56.9%]

Correct runs: 40 of 40. Effects from a branch that never retired: 0. Section 10 is the same against a real model.

10. The same, against a real model

claude-sonnet -5, 15 rounds, one run of each configuration per round. One call per reply is enforced with the API's `disable_parallel_tool_use`: asked in the prompt alone, the model batched its calls anyway.

Configuration	Replies	Calls/reply	Wall clock	Cost/run	Correct
One call, no cache (before)	11.0	1.00	16151 ms	\$0.0636	15 of 15
One call, cached	11.0	1.00	17350 ms	\$0.0173	15 of 15
Several calls, no cache	5.0	2.50	10804 ms	\$0.0361	15 of 15
Several calls, cached (after)	5.0	2.50	10431 ms	\$0.0142	15 of 15
No guidance, cached	5.0	2.50	10254 ms	\$0.0141	15 of 15

Change	Wall clock saved	Cost saved
Caching alone	-7.3% [-17.5%, +1.2%]	+72.7% [+71.9%, +73.6%]
Several calls per reply	+39.7% [+34.4%, +45.1%]	+18.3% [+14.9%, +21.6%]
Before to after	+35.4% [+31.9%, +39.0%]	+77.7% [+77.0%, +78.3%]
Parallel nodes	+60.2% [+57.6%, +62.6%]	--

Spend \$2.33. Effects from a branch that never retired: 0.

11. Pull the plug: what a crash sends twice

A billing run with 7 effects, killed 14 times -- just before each request reached the upstream, and just after each took effect but before its reply came back -- and restarted the way each system restarts. LangGraph 1.2.11; no model and no network.

System	Exact	Stopped for a human	Sent twice	Extra effects
Plain async loop	1	0	13	49
LangGraph, node per customer	4	0	10	13
LangGraph, @task per call	7	0	7	7
SpecuNode	0	14	0	0
SpecuNode with reconcile	14	0	0	0

A lost reply is the crash that double-charges. SpecuNode sent nothing twice in any of them; without a way to ask the upstream it stops for a human, and with a reconcile per tool it finished every one, nothing sent twice and nothing missing.